



國立高雄科技大學

National Kaohsiung University of Science and Technology

AI AGENT (人工智慧代理)

AI 協作 - 軟體開發生命週期

Instructor: Shih-Huan Tseng

shtseng@nkust.edu.tw

Department of Computer and Communication Engineering

National Kaohsiung University of Science and Technology



AI 協作軟體開發生命週期 (AI-assisted SDLC)

帶領 AI 團隊的高效開發流程

1. 開發思維的轉變

- 傳統開發模式

- 手動編寫程式碼，花費大量時間在語法與 API 除錯上。

- AI 協作開發模式

- 工程師角色轉變為「AI 專案經理與審查員」。
- 專注於架構設計、規則制定與成果驗證 (Testing & Verification) 。

2. AI-assisted SDLC 藍圖

Rules (規範) —> Alignment (需求對齊) —> Plan (計畫制定) —> Task (任務追蹤)
|
Evolution (沉澱進化) <— Verification (驗證交付) <— Execution (平行執行) ↓

1. 規範定義 (Rules) : `AGENTS.md`
2. 需求對齊 (Alignment) : 互動問答收斂需求
3. 計畫制定 (Plan) : `implementation_plan.md`
4. 進度追蹤 (Task) : `task.md`
5. 平行執行 (Execution) : 背景任務 & Subagents
6. 驗證交付 (Verification) : `walkthrough.md`
7. 沉澱進化 (Evolution) : Custom Skills & `/learn`

3. 【實作一】 Rules 制定專案鐵律

- 全域與專案規則的分野：
 - 全域規則 (Global Rules)：存於個人設定，適用於所有專案。
 - 專案區域規則 (Workspace Rules)：存於專案根目錄
`.agents/AGENTS.md`，僅對此專案生效。
- 專案區域規則實作：

- 使用 Python 開發程式時，必須使用 `uv` 建立虛擬環境。
- 嚴禁使用 base 環境或直接用 pip，維持環境純淨。

4. 【實作二】 Requirement & Plan

- 人機需求對齊 (Alignment) :
 - 寫 code 前的關鍵步驟：釐清「使用者真正要什麼」（如使用 `/grill-me` 指令）。
- 計畫書 (`implementation_plan.md`) 三大核心：
 1. Open Questions：向使用者釐清並對齊模糊或未定義的需求。
 2. Proposed Changes：條列預估異動的檔案與架構。
 3. Verification Plan：明訂功能驗證方法。

5. 【實作三】Task Tracking 進度看板

- 什麼是 Task Tracking ?
 - 計畫被核准後，自動生成的 `task.md` 任務清單。
- 追蹤格式：
 - `[ ]` 未完成任務
 - `[/]` 正在進行中的任務 (Progressing)
 - `[x]` 已完成並驗證的任務

6. 【實作四】 Execution 背景執行與分工

- 背景任務 (Background Tasks)
 - 安裝套件、執行長測試時移至背景，不阻塞與 Agent 的對話。
- 子代理人 (Subagents)
 - 呼叫專職 Agent (如： `research`) 平行搜尋論文或調用特定 API。

7. 【實作五】 Verification 嚴謹的驗證報告

- 什麼是 Verification ?
 - 程式寫完後，透過 `walkthrough.md` 交付的成果報告。
- 驗證報告內容：
 - 修改檔案的 **Diff** (差異對比)。
 - 自動化與手動測試的**執行結果** (Pass/Fail)。
 - 產出檔案 (如：[merged.pdf]
(file:///d:/Antigravity%20Codes/PDF%20Merge/pdf/merged.pdf)) 的直接連結與截圖。

8. 【實作六】Skills 最佳實踐的提煉

- 什麼是 Skills ?
 - 當一個工作流測試成功後，將其打包供未來直接使用。
- 目錄結構：

```
.agents/skills/thsr-ticket-parser/  
├── SKILL.md           # YAML 屬性與使用指令說明  
└── scripts/          # 實作程式碼 (Python)
```

9. 結語：成為 AI 時代的架構師

- 程式碼撰寫成本已趨近於零。
- 真正的門檻在於：規範、對齊、計畫與驗證。
- 掌握 Rules → Alignment → Plan → Tasks → Walkthrough → Skills 閉環，成為引領 AI 團隊的架構大師。